

ISC_permutation_general_block

July 18, 2025

```
[1]: ##modules
    %%matplotlib widget
    %matplotlib inline
    #
    %%matplotlib qt
    import mne
    import numpy as np
    import matplotlib
    # Establecer un backend interactivo, como 'Qt5Agg', 'GTK3Agg', etc.
    # Esto depende de los backends disponibles en tu sistema.

    import matplotlib.pyplot as plt

    %matplotlib.use('TkAgg') # Asegúrate de que este backend está instalado.

    import pandas as pd
    import os
    import sys

    from mne.preprocessing import ICA, corrmap, create_ecg_epochs, create_eog_epochs

    from os.path import join as pathjoin
    from time import time

    from pathlib import Path

    from autoreject import AutoReject

    # aplicar la acf EN epochs

    from statsmodels.tsa.stattools import acf

    from scipy.stats import permutation_test

    sys.path.append(r"G:\apuntes_mne\code_MOUS\scripts_functions_created")
```

```
# Ahora importa la función
from print5 import print5
```

[2]:

```
try:
    # Si se ejecuta como SCRIPT .py: usar __file__
    sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..
↳")))
except NameError:
    # Si se ejecuta como NOTEBOOK Jupyter: usar path relativo
    sys.path.append("..") # sube un nivel desde la carpeta actual del notebook

# --- Configuración dinámica de rutas ---
from get_paths_MOUS import get_paths_MOUS

# Parámetros editables
disco = "g"
modality = "visual"
layer_script = "block"
subj = "sub-V1001"
#subj = sys.argv[1] ## name of participant list

# Generar variables automáticamente
path_dict = get_paths_MOUS(disco=disco, modality=modality,
↳layer_script=layer_script, subj=subj)
globals().update(path_dict)

# Mostrar todos los paths generados
print("\n Rutas generadas:")
for k, v in path_dict.items():
    print(f"{k:<20} → {v}")
```

```
Rutas generadas:
datadir          → g:\MOUS_204\MOUS_visual
general_datadir  → g:\MOUS_204
output_preproc   → g:\MOUS_204\MOUS_visual\output_preproc
mri_dir          → g:\MOUS_204\sub-V1001\anat
meg_dir          → g:\MOUS_204\sub-V1001\meg
preproc_path     → g:\MOUS_204\MOUS_visual\output_preproc\preproc_block
channels_structure_path → g:\MOUS_204\channels_structure
epochs_path      →
g:\MOUS_204\MOUS_visual\output_preproc\preproc_block\epochs_block
ICA_path         →
g:\MOUS_204\MOUS_visual\output_preproc\preproc_block\ICA_block
epochs_clean_path →
```

```

g:\MOUS_204\MOUS_visual\output_preproc\preproc_block\epochs_clean_block
evoked_path          →
g:\MOUS_204\MOUS_visual\output_preproc\preproc_block\evoked_block
source_path          → g:\MOUS_204\MOUS_visual\output_source\source_block
raw_hsp_path         →
g:\MOUS_204\MOUS_visual\output_source\source_block\raw_hsp
fwd_path             → g:\MOUS_204\MOUS_visual\output_source\source_block\fwd
inverse_path         →
g:\MOUS_204\MOUS_visual\output_source\source_block\inverse
output_analysis      → g:\MOUS_204\MOUS_visual\output_analysis
analysis_path        → g:\MOUS_204\MOUS_visual\output_analysis\analysis_block
ACW_path             →
g:\MOUS_204\MOUS_visual\output_analysis\analysis_block\acw_block
PLE_path            →
g:\MOUS_204\MOUS_visual\output_analysis\analysis_block\PLE_block
ISC_path            →
g:\MOUS_204\MOUS_visual\output_analysis\analysis_block\ISC_block
ISC_block_path       →
g:\MOUS_204\MOUS_visual\output_analysis\analysis_block\ISC_block

```

```

[3]: def print5(*args):
        print(*(f"{x:.5f}" if isinstance(x, float) else x for x in args))

channels = pd.read_csv(channels_structure_path / f"channels_mag_{modality}.csv")
channels_mag=channels[channels[f"canal_efectivo_{modality}"].
↳notna()][f"canal_efectivo_{modality}"]

channels_mag=channels_mag.tolist()
print5(channels_mag)
indice_channels_efectivos = channels[channels[f"canal_efectivo_{modality}"].
↳notna()]["indice"]
indice_channels_efectivos=indice_channels_efectivos.tolist()
# del channels

# del channels

```

```

['MLC11-4304', 'MLC12-4304', 'MLC13-4304', 'MLC14-4304', 'MLC15-4304',
'MLC16-4304', 'MLC17-4304', 'MLC21-4304', 'MLC22-4304', 'MLC23-4304',
'MLC24-4304', 'MLC25-4304', 'MLC31-4304', 'MLC32-4304', 'MLC41-4304',
'MLC42-4304', 'MLC51-4304', 'MLC52-4304', 'MLC53-4304', 'MLC54-4304',
'MLC55-4304', 'MLC61-4304', 'MLC62-4304', 'MLC63-4304', 'MLF11-4304',
'MLF12-4304', 'MLF13-4304', 'MLF14-4304', 'MLF21-4304', 'MLF22-4304',
'MLF23-4304', 'MLF24-4304', 'MLF25-4304', 'MLF31-4304', 'MLF32-4304',
'MLF33-4304', 'MLF34-4304', 'MLF35-4304', 'MLF41-4304', 'MLF42-4304',
'MLF43-4304', 'MLF44-4304', 'MLF45-4304', 'MLF46-4304', 'MLF51-4304',
'MLF52-4304', 'MLF53-4304', 'MLF54-4304', 'MLF55-4304', 'MLF56-4304',
'MLF61-4304', 'MLF63-4304', 'MLF64-4304', 'MLF65-4304', 'MLF66-4304',
'MLF67-4304', 'ML011-4304', 'ML012-4304', 'ML013-4304', 'ML014-4304',

```

'ML021-4304', 'ML022-4304', 'ML023-4304', 'ML024-4304', 'ML031-4304',
 'ML032-4304', 'ML033-4304', 'ML034-4304', 'ML041-4304', 'ML042-4304',
 'ML043-4304', 'ML044-4304', 'ML051-4304', 'ML052-4304', 'ML053-4304',
 'MLP11-4304', 'MLP12-4304', 'MLP21-4304', 'MLP22-4304', 'MLP23-4304',
 'MLP31-4304', 'MLP32-4304', 'MLP33-4304', 'MLP34-4304', 'MLP35-4304',
 'MLP41-4304', 'MLP42-4304', 'MLP43-4304', 'MLP44-4304', 'MLP45-4304',
 'MLP51-4304', 'MLP52-4304', 'MLP53-4304', 'MLP54-4304', 'MLP55-4304',
 'MLP56-4304', 'MLP57-4304', 'MLT11-4304', 'MLT12-4304', 'MLT13-4304',
 'MLT14-4304', 'MLT15-4304', 'MLT16-4304', 'MLT21-4304', 'MLT22-4304',
 'MLT23-4304', 'MLT24-4304', 'MLT25-4304', 'MLT26-4304', 'MLT27-4304',
 'MLT31-4304', 'MLT32-4304', 'MLT33-4304', 'MLT34-4304', 'MLT35-4304',
 'MLT36-4304', 'MLT41-4304', 'MLT42-4304', 'MLT43-4304', 'MLT44-4304',
 'MLT45-4304', 'MLT46-4304', 'MLT47-4304', 'MLT51-4304', 'MLT52-4304',
 'MLT53-4304', 'MLT54-4304', 'MLT55-4304', 'MLT56-4304', 'MLT57-4304',
 'MRC11-4304', 'MRC12-4304', 'MRC13-4304', 'MRC14-4304', 'MRC15-4304',
 'MRC16-4304', 'MRC17-4304', 'MRC21-4304', 'MRC22-4304', 'MRC23-4304',
 'MRC24-4304', 'MRC25-4304', 'MRC31-4304', 'MRC32-4304', 'MRC41-4304',
 'MRC42-4304', 'MRC51-4304', 'MRC52-4304', 'MRC53-4304', 'MRC54-4304',
 'MRC55-4304', 'MRC61-4304', 'MRC62-4304', 'MRC63-4304', 'MRF11-4304',
 'MRF12-4304', 'MRF13-4304', 'MRF14-4304', 'MRF21-4304', 'MRF22-4304',
 'MRF23-4304', 'MRF24-4304', 'MRF25-4304', 'MRF31-4304', 'MRF32-4304',
 'MRF33-4304', 'MRF34-4304', 'MRF35-4304', 'MRF41-4304', 'MRF42-4304',
 'MRF43-4304', 'MRF44-4304', 'MRF45-4304', 'MRF46-4304', 'MRF51-4304',
 'MRF52-4304', 'MRF53-4304', 'MRF54-4304', 'MRF55-4304', 'MRF56-4304',
 'MRF61-4304', 'MRF62-4304', 'MRF63-4304', 'MRF64-4304', 'MRF65-4304',
 'MRF66-4304', 'MRF67-4304', 'MR011-4304', 'MR012-4304', 'MR013-4304',
 'MR014-4304', 'MR021-4304', 'MR022-4304', 'MR023-4304', 'MR024-4304',
 'MR031-4304', 'MR032-4304', 'MR033-4304', 'MR034-4304', 'MR041-4304',
 'MR042-4304', 'MR043-4304', 'MR044-4304', 'MR051-4304', 'MR052-4304',
 'MR053-4304', 'MRP11-4304', 'MRP12-4304', 'MRP21-4304', 'MRP22-4304',
 'MRP23-4304', 'MRP31-4304', 'MRP32-4304', 'MRP33-4304', 'MRP34-4304',
 'MRP35-4304', 'MRP41-4304', 'MRP42-4304', 'MRP43-4304', 'MRP44-4304',
 'MRP45-4304', 'MRP51-4304', 'MRP52-4304', 'MRP53-4304', 'MRP54-4304',
 'MRP55-4304', 'MRP56-4304', 'MRP57-4304', 'MRT11-4304', 'MRT12-4304',
 'MRT13-4304', 'MRT14-4304', 'MRT15-4304', 'MRT16-4304', 'MRT21-4304',
 'MRT22-4304', 'MRT23-4304', 'MRT24-4304', 'MRT25-4304', 'MRT26-4304',
 'MRT27-4304', 'MRT31-4304', 'MRT32-4304', 'MRT33-4304', 'MRT34-4304',
 'MRT35-4304', 'MRT36-4304', 'MRT37-4304', 'MRT41-4304', 'MRT42-4304',
 'MRT43-4304', 'MRT44-4304', 'MRT45-4304', 'MRT46-4304', 'MRT47-4304',
 'MRT51-4304', 'MRT52-4304', 'MRT53-4304', 'MRT54-4304', 'MRT55-4304',
 'MRT56-4304', 'MRT57-4304', 'MZC01-4304', 'MZC02-4304', 'MZC03-4304',
 'MZC04-4304', 'MZF01-4304', 'MZF02-4304', 'MZF03-4304', 'MZ001-4304',
 'MZ002-4304', 'MZ003-4304', 'MZO01-4304']

```

[4]: #lectura de los diccionarios ISC_zinnen y ISC_woorden
dict_woorden_block = pd.read_pickle(ISC_path / f"dict_woorden_block.pkl")
dict_zinnen_block = pd.read_pickle(ISC_path / f"dict_zinnen_block.pkl")
  
```

```
[5]: # ##channels significant adjusted in each condition
# numbers_channels_woorden=
↳dict_woorden_block["significant_channels_adjusted_woorden"]
# numbers_channels_zinnen=
↳dict_zinnen_block["significant_channels_adjusted_zinnen"]
# print("numbers_channels_woorden", numbers_channels_woorden,
↳"len=", len(numbers_channels_woorden))

# print("numbers_channels_zinnen", numbers_channels_zinnen, "len=",
↳len(numbers_channels_zinnen))
```

```
[6]: ##channels significant adjusted in each condition
numbers_channels_woorden=
↳dict_woorden_block["significant_channels_adjusted_woorden"]
numbers_channels_zinnen=
↳dict_zinnen_block["significant_channels_adjusted_zinnen"]
print("numbers_channels_woorden", numbers_channels_woorden,
↳"numbers_channels_zinnen", numbers_channels_zinnen)
print("len(numbers_channels_woorden)=", len(numbers_channels_woorden),
↳"len(numbers_channels_zinnen)=", len(numbers_channels_zinnen))

names_channels_woorden=[channels_mag[i] for i in numbers_channels_woorden]
names_channels_zinnen=[channels_mag[i] for i in numbers_channels_zinnen]

print("names_channels_woorden", names_channels_woorden)
print("names_channels_zinnen", names_channels_zinnen)

#establecimiento de canales palabras, canales frases y canales mixtos
numbers_channels_only_woorden = list(
    set(numbers_channels_woorden) - set(numbers_channels_zinnen)
)

numbers_channels_only_zinnen = list(
    set(numbers_channels_zinnen) - set(numbers_channels_woorden)
)

number_channels_intersection = list(
    set(numbers_channels_woorden) & set(numbers_channels_zinnen)
)

print(f"len(significant_channels_only_woorden):
↳{len(numbers_channels_only_woorden)},\
    len(significant_channels_only_zinnen):
↳{len(numbers_channels_only_zinnen)},\
    len(significant_channels_intersection)
↳{len(number_channels_intersection)}")
```

```
#create list of names of channels
```

```
names_channels_only_woorden = [channels_mag[i] for i in_
    ↪numbers_channels_only_woorden]
names_channels_only_zinnen = [channels_mag[i] for i in_
    ↪numbers_channels_only_zinnen]
names_channels_intersection = [channels_mag[i] for i in_
    ↪number_channels_intersection]
```

```
numbers_channels_woorden [ 1  14  23  46  52  56  58  59  60  63  67  71  81
82 83 84 86 87
    88 89 90 92 93 94 112 118 122 133 140 141 144 145 150 180 184 192
    201 210 213 215 220 221 224 237 240 252 254 256 269 270]
numbers_channels_zinnen [ 10  13  29  31  32  36  37  40  41  45  46  56  60  72
79 82 83 84
    86 87 88 89 94 105 106 112 117 118 123 135 136 140 141 143 144 145
    155 167 173 199 200 211 212 215 216 217 219 223 228 229 230 234 235 242
    243 244 250 251 256]
len(numbers_channels_woorden)= 50 len(numbers_channels_zinnen)= 59
names_channels_woorden ['MLC12-4304', 'MLC41-4304', 'MLC63-4304', 'MLF53-4304',
'MLF64-4304', 'ML011-4304', 'ML013-4304', 'ML014-4304', 'ML021-4304',
'ML024-4304', 'ML034-4304', 'ML044-4304', 'MLP32-4304', 'MLP33-4304',
'MLP34-4304', 'MLP35-4304', 'MLP42-4304', 'MLP43-4304', 'MLP44-4304',
'MLP45-4304', 'MLP51-4304', 'MLP53-4304', 'MLP54-4304', 'MLP55-4304',
'MLT33-4304', 'MLT43-4304', 'MLT47-4304', 'MRC14-4304', 'MRC24-4304',
'MRC25-4304', 'MRC41-4304', 'MRC42-4304', 'MRC55-4304', 'MRF61-4304',
'MRF65-4304', 'MRO22-4304', 'MRO43-4304', 'MRP23-4304', 'MRP33-4304',
'MRP35-4304', 'MRP45-4304', 'MRP51-4304', 'MRP54-4304', 'MRT24-4304',
'MRT27-4304', 'MRT45-4304', 'MRT47-4304', 'MRT52-4304', 'MZ001-4304',
'MZ002-4304']
names_channels_zinnen ['MLC24-4304', 'MLC32-4304', 'MLF22-4304', 'MLF24-4304',
'MLF25-4304', 'MLF34-4304', 'MLF35-4304', 'MLF43-4304', 'MLF44-4304',
'MLF52-4304', 'MLF53-4304', 'ML011-4304', 'ML021-4304', 'ML051-4304',
'MLP23-4304', 'MLP33-4304', 'MLP34-4304', 'MLP35-4304', 'MLP42-4304',
'MLP43-4304', 'MLP44-4304', 'MLP45-4304', 'MLP55-4304', 'MLT23-4304',
'MLT24-4304', 'MLT33-4304', 'MLT42-4304', 'MLT43-4304', 'MLT51-4304',
'MRC16-4304', 'MRC17-4304', 'MRC24-4304', 'MRC25-4304', 'MRC32-4304',
'MRC41-4304', 'MRC42-4304', 'MRF12-4304', 'MRF35-4304', 'MRF46-4304',
'MRO41-4304', 'MRO42-4304', 'MRP31-4304', 'MRP32-4304', 'MRP35-4304',
'MRP41-4304', 'MRP42-4304', 'MRP44-4304', 'MRP53-4304', 'MRT11-4304',
'MRT12-4304', 'MRT13-4304', 'MRT21-4304', 'MRT22-4304', 'MRT32-4304',
'MRT33-4304', 'MRT34-4304', 'MRT43-4304', 'MRT44-4304', 'MRT52-4304']
len(significant_channels_only_woorden): 31,
len(significant_channels_only_zinnen): 40,
len(significant_channels_intersection) 19
```

```
[7]: iscs_bootstrap_woorden=dict_woorden_block["iscs_bootstrap_woorden"]
iscs_bootstrap_zinnen=dict_zinnen_block["iscs_bootstrap_zinnen"]

TRW= iscs_bootstrap_zinnen - iscs_bootstrap_woorden
```

0.1 PLOTS for ISCS

```
[8]: ##plots for ISC

#read epochs to build evoked
#read epochs to build evoked
epochs_zinnen=mne.read_epochs(epochs_clean_path /
    ↳f"{subj}_epochs_zinnen_block-epo.fif")
len(epochs_zinnen.pick("mag", exclude="bads").ch_names)

evoked_zinnen=epochs_zinnen.average()
## evoked_zinnen es una lista

##cojo el primer elemento, solo tengo una lista
evoked_zinnen=evoked_zinnen

info=evoked_zinnen.info
del epochs_zinnen
del evoked_zinnen
del evoked_zinnen
```

Reading g:\MOUS_204\MOUS_visual\output_preproc\preproc_block\epochs_clean_block\sub-V1001_epochs_zinnen_block-epo.fif ...

Found the data of interest:

t = 0.00 ... 60000.00 ms

0 CTF compensation matrices available

Not setting metadata

24 matching events found

No baseline correction applied

0 projection items activated

```
[9]: #Possible masks

mask_zinnen= np.zeros(iscs_bootstrap_woorden.shape, dtype=bool)
for electrode in numbers_channels_zinnen:
    mask_zinnen[electrode] = True

mask_woorden= np.zeros(iscs_bootstrap_woorden.shape, dtype=bool)
for electrode in numbers_channels_woorden:
    mask_woorden[electrode] = True

mask_zinnen_only= np.zeros(iscs_bootstrap_woorden.shape, dtype=bool)
```

```

for electrode in numbers_channels_only_zinnen:
    mask_zinnen_only[electrode] = True

mask_woorden_only= np.zeros(iscs_bootstrap_woorden.shape, dtype=bool)
for electrode in numbers_channels_only_woorden:
    mask_woorden_only[electrode] = True

mask_intersection= np.zeros(iscs_bootstrap_woorden.shape, dtype=bool)
for electrode in number_channels_intersection:
    mask_intersection[electrode] = True

##in data i use the value of t_obs_zinnen
#pos takes the information from evoked.info object
#mask takes the mask_zinnen object
#in t stat i don't change it because the p value will be calculated with
↳permutations, so no problem when using a parametric test,
# even if the distribution is non parametric
dict_masks= {"mask_zinnen":mask_zinnen,
             "mask_woorden":mask_woorden,
             "mask_intersection": mask_intersection,
             "mask_zinnen_only": mask_zinnen_only,
             "mask_woorden_only": mask_woorden_only,
             }

iscs_bootstrap={"iscs_bootstrap_woorden":iscs_bootstrap_woorden,
               "iscs_bootstrap_zinnen":iscs_bootstrap_zinnen}

for iscs_name, values in iscs_bootstrap.items():
    print(f"ISCs are: {iscs_name}")

    # Seleccionamos la máscara correcta
    if iscs_name == "iscs_bootstrap_zinnen":
        mask = mask_zinnen
    elif iscs_name == "iscs_bootstrap_woorden":
        mask = mask_woorden

    # Creamos figura y eje
    fig, ax = plt.subplots(figsize=(6, 6))

    # Dibujamos topomap sobre ese eje
    im, _ = mne.viz.plot_topomap(
        data=values,
        pos=info,
        mask=mask,
        axes=ax,

```



```

cmap='RdBu_r',
vlim=(-np.max(np.abs(values)), np.max(np.abs(values))),
mask_params=dict(marker='x', markerfacecolor='green', markersize=10),
contours=0,
show=False # lo mostramos al final
)

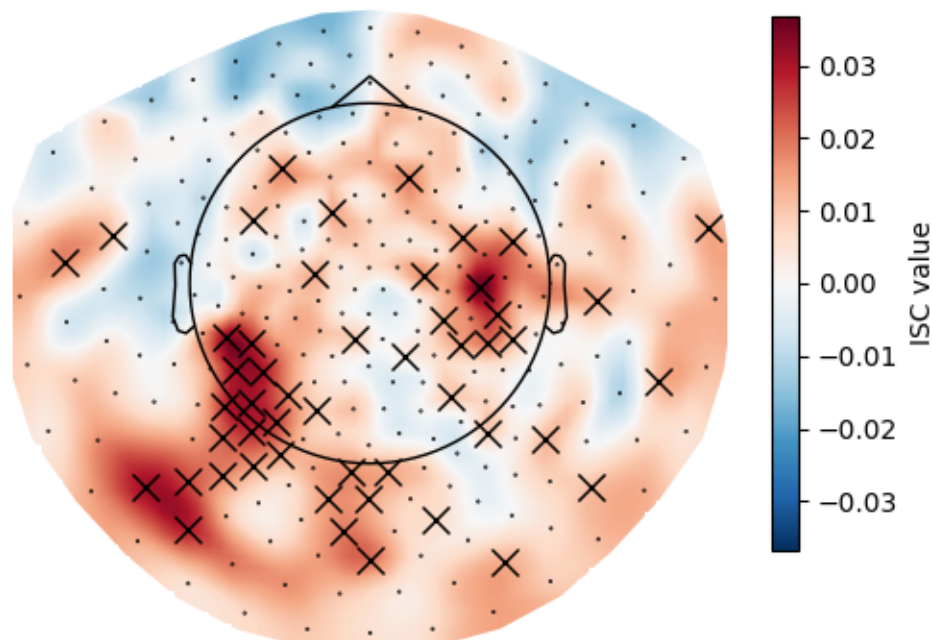
# Añadir título y colorbar
fig.suptitle(f"Values of {iscs_name}", fontsize=24)
cbar = fig.colorbar(im, ax=ax, shrink=0.6)
cbar.set_label("ISC value")
del mask
# Save the figure with a specific filename
# fig.savefig(ISC_path / f"{mask_name}_words.png", dpi=300,
↳ bbox_inches='tight')

```

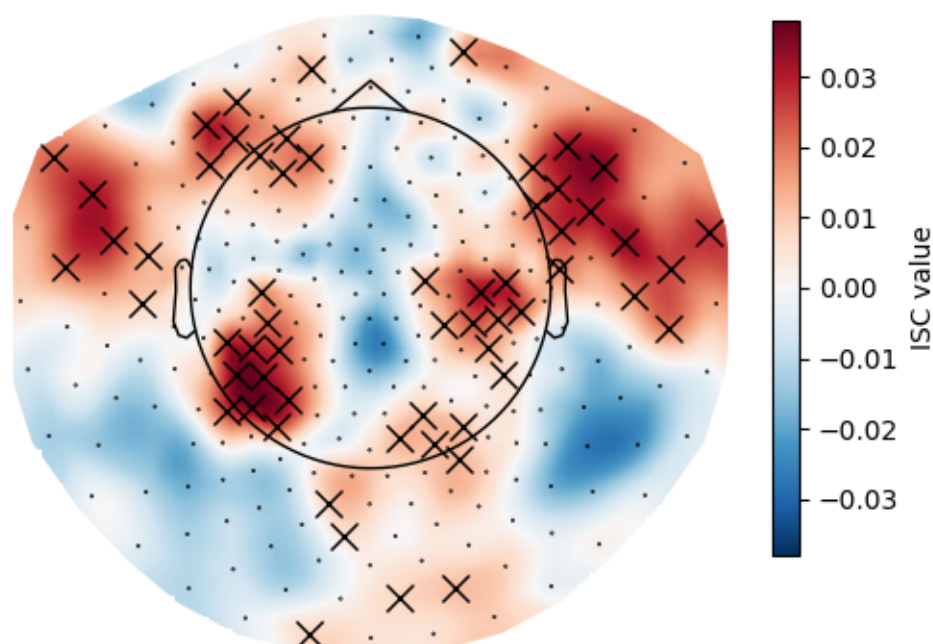
ISCs are: iscs_bootstrap_woorden

ISCs are: iscs_bootstrap_zinnen

Values of iscs_bootstrap_woorden



Values of iscs_bootstrap_zinnen



```
[16]: ##plots of intersection

mask= mask_intersection
fig, ax = plt.subplots(figsize=(6, 6)) # Tamaño explícito

im, _ = mne.viz.plot_topomap(
    data=np.zeros(iscs_bootstrap_woorden.shape),
    pos=info,
    mask=mask,
    axes=ax, # usar el eje explícito
    cmap='RdBu_r',
    vlim=(-np.max(np.abs(values)), np.max(np.abs(values))),
    mask_params=dict(marker='x', markerfacecolor='green', markersize=10),
    contours=0,
    show=False
)
```

```

fig.suptitle("Plot for intersection", fontsize=34)

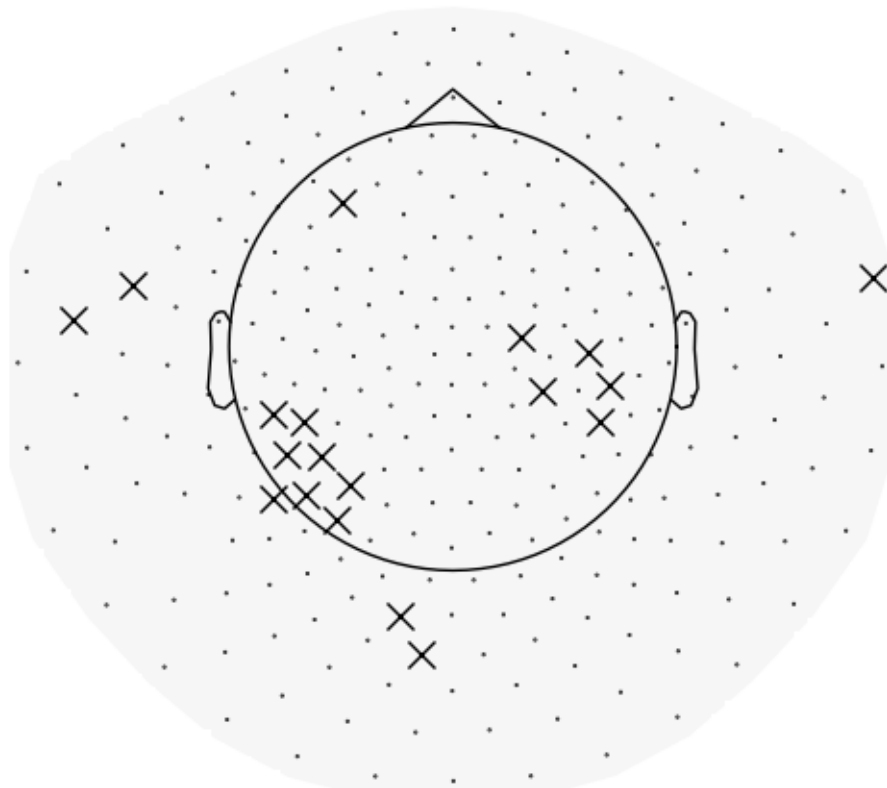
# cbar = fig.colorbar(im, ax=ax, shrink=0.6)
# cbar.set_label("Values")

plt.show()

del mask

```

Plot for intersection



```

[17]: ##plot for only zinnen
mask = mask_zinnen_only

# Crear figura y eje con tamaño definido
fig, ax = plt.subplots(figsize=(6, 6))

```

```

# Dibujar el topomap sobre ese eje
im, _ = mne.viz.plot_topomap(
    data=iscs_bootstrap_zinnen,
    pos=info,
    mask=mask,
    axes=ax, # eje explícito
    cmap='RdBu_r',
    vlim=(-np.max(np.abs(values)), np.max(np.abs(values))),
    mask_params=dict(marker='x', markerfacecolor='green', markersize=10),
    contours=0,
    show=False # mostrar después
)

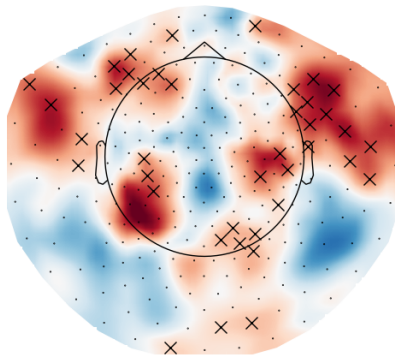
# Título
fig.suptitle("Values of iscs_sentences with channels only sentences",
             ↪fontsize=34)

# Mostrar figura
plt.show()

# Eliminar máscara para evitar conflictos posteriores
del mask

```

Values of iscs_sentences with channels only sentences



0.2 Posibles comparaciones estadísticas

len(significant_channels_only_woorden): 25, len(significant_channels_only_zinnen): 92,
len(significant_channels_intersection) 88

Primer problema, el numero de canales zinnen es claramente superior al de woorden, big problem, aunque la interseccion es alta, tal como esperaríamos. Seguramente esto sea porque hay ruido

- comparación ACW-50 en zinnen entre estos canales zinnen y canales woorden + comapracion

en woorden de lo mismo

- comparación entre solo zinnen con intersection en woorden y en zinnen (aunque ojo, esto es básicamente aceptar que los canales solo Zinnen son ruido)
- comparación en el promedio general de acw en ambas condiciones de los canales zinnen vs promedio general de canales woorden
- comparación de la intersección vs canales no significativos: no sé muy bien como interpretar esto
-

0.3 general comparisons of ISC

```
[12]: ##selección de valores
      ##matrices de valores de slope, divididas por condicion y tipo de channel

      ## in this case we are going to use the values of all the channels, as these
      ↪are the values of ISC, wich are created comapring subjects
      X_zinnen = iscs_bootstrap_zinnen

      X_woorden = iscs_bootstrap_woorden

      print(f"X_zinnen.shape is {X_zinnen.shape}, X_woorden.shape is {X_woorden.
      ↪shape}")
```

X_zinnen.shape is (273,), X_woorden.shape is (273,)

```
[13]: ##dependent condition
      def paired_statistic(diff, _):
          return np.mean(diff)

      # Crear el diccionario con nombres descriptivos
      data_dict = {
          "X_zinnen": X_zinnen,
          "X_woorden": X_woorden}

      print("Differences in ISC between conditions")
      x= data_dict[f"X_zinnen"]
      ##notice that in this comparison y is ch_woorden_mean or ch_intersection_mean
      y= data_dict[f"X_woorden"]

      diff=x-y
      # Permutation test
      res = permutation_test(
          (diff, np.zeros_like(diff)),
```

```

#
statistic=paired_statistic,
vectorized=False,
n_resamples=10000,
alternative='Greater',
random_state=42
)

print5(f"Statistic value: {res.statistic}")
print5(f"p-value: {res.pvalue}")

```

Differences in ISC between conditions
Statistic value: -0.0017132490338651676
p-value: 0.9686031396860314

[14]: y

```

[14]: array([ 1.76675696e-03,  1.40069153e-02, -5.74605154e-03,  4.43519082e-03,
        -7.50581398e-03,  5.14279511e-03, -1.39975949e-03,  8.84689771e-03,
         5.64380325e-04,  4.50785872e-03,  1.17450133e-02,  1.66915967e-02,
         9.49811034e-03,  1.01045960e-02,  1.35463045e-02,  1.39262475e-02,
         9.03697277e-03,  3.01681807e-03,  9.35346544e-03,  1.12834198e-02,
         4.21688248e-03,  2.37630968e-03,  7.42545386e-04,  9.41867602e-03,
        -1.58776116e-02, -1.17306350e-02, -1.74218667e-02, -4.22533008e-04,
        -1.12069438e-02, -5.59847714e-03, -9.21578708e-03, -1.32697415e-02,
        -6.92889349e-04, -1.06682067e-02, -1.57815397e-02,  4.43848613e-03,
         3.11958510e-03, -4.12496986e-03, -8.11493936e-03, -1.21958172e-02,
         6.24105075e-04,  3.61497863e-03, -3.82898215e-04, -9.10234325e-03,
         5.43578172e-03,  2.23624293e-03,  1.71191831e-02,  9.46507019e-03,
         4.35218655e-03, -6.99332833e-03,  1.03022459e-02,  7.78863189e-03,
         9.24119830e-03,  2.19735305e-03,  6.87589183e-04, -3.35549711e-03,
         1.62524894e-02,  6.61775251e-03,  1.61756627e-02,  2.23540804e-02,
         1.85990627e-02,  9.19845398e-03,  3.50639087e-03,  1.82440970e-02,
         1.54269357e-02,  5.05814896e-03,  9.39360037e-03,  2.31294228e-02,
         1.10736024e-02,  1.68124672e-02,  1.94596431e-02,  2.95796699e-02,
         1.01593795e-02,  8.09753029e-03,  7.06683147e-03,  1.73286753e-03,
         1.08937861e-02,  1.06480998e-02,  6.45667260e-03,  1.57714753e-02,
         7.28954192e-03,  8.69534913e-03,  1.63450557e-02,  2.88389946e-02,
         2.73946185e-02,  1.02290008e-02,  2.48834372e-02,  3.01531909e-02,
         3.23230982e-02,  3.35731659e-02,  1.58664451e-02,  1.43541157e-02,
         1.59201225e-02,  2.82672725e-02,  2.90276929e-02,  1.80948894e-02,
         1.60590108e-02, -4.42049060e-03, -4.90641278e-03, -6.93052873e-03,
        -7.40768202e-03,  4.27685019e-03,  1.16588122e-02,  8.95056238e-04,
        -7.45369634e-03, -1.26949040e-02, -9.41142953e-03, -9.59746999e-04,
         1.52100812e-03,  1.45235776e-02, -1.27072664e-02,  1.40269091e-03,
         8.36739895e-03, -6.20848674e-03,  2.90934388e-03,  6.08784575e-03,
         7.79258894e-03,  2.37274806e-03,  1.74941780e-02, -7.06517685e-03,
         9.72257799e-03,  1.70292333e-02,  3.11878027e-02, -6.95707817e-03,

```

3.88535189e-03, 1.31074259e-02, 6.83099582e-03, 4.07642877e-03,
 1.63071347e-02, 1.49939291e-02, 7.68107552e-03, 1.21870049e-02,
 1.22627889e-02, 1.57103790e-02, 2.34212118e-02, 2.40789225e-02,
 1.15460627e-02, 2.95721376e-03, 3.63400809e-03, 1.53629429e-02,
 3.66739084e-02, 2.57549343e-02, 1.43322969e-02, 1.85244968e-02,
 1.76189263e-02, 1.30200659e-02, 5.06317328e-03, 1.10408292e-02,
 2.91213993e-03, 1.76979070e-03, 3.50345483e-03, -4.61235595e-03,
 -2.86970048e-03, -4.92256419e-04, 1.18545425e-02, 1.00372185e-02,
 5.23538439e-03, -9.69535073e-03, 5.93086451e-03, 1.33375333e-03,
 1.91933513e-03, 8.57077945e-03, -9.62857316e-03, 4.00008240e-03,
 -2.65164258e-03, -2.75675637e-03, -4.97910003e-03, -1.17878161e-02,
 1.21333437e-02, 8.80480991e-03, 8.05781323e-03, -1.06852630e-02,
 -2.91102549e-03, -9.06650641e-03, 5.35924313e-03, 8.86745475e-03,
 8.06716775e-03, -1.77476646e-03, 2.30762282e-03, 3.00807663e-04,
 1.77506310e-02, 8.22204031e-03, 1.88483161e-03, 5.80744462e-03,
 1.73333035e-02, 1.70920654e-02, 1.87573561e-02, 8.46615406e-03,
 3.77883913e-03, -2.60371182e-03, 1.62252904e-03, 8.79748115e-03,
 6.81245042e-03, -2.30871546e-03, 2.93503705e-03, 4.87337031e-03,
 2.74758983e-03, -1.59807246e-05, 8.56436845e-03, 4.12876158e-03,
 1.06501754e-02, 1.25441319e-02, 5.49703962e-03, 6.42966145e-03,
 1.50028418e-02, 8.72517352e-03, -5.36745770e-03, 6.36080715e-03,
 -3.00048680e-03, -2.04723013e-03, 1.45940242e-02, -4.50029595e-03,
 1.36541600e-04, 1.06441204e-02, 3.07716954e-03, 2.01912789e-02,
 -3.57114855e-03, -1.98404075e-03, 7.74202233e-03, 3.75518162e-03,
 1.26714121e-02, 1.36232276e-02, 8.57961270e-03, -5.52983987e-03,
 1.02817736e-02, 6.97042312e-04, -8.55843660e-04, 2.62869909e-03,
 -1.22113297e-03, 3.73150178e-03, 8.84775945e-03, 3.22696597e-03,
 1.20632870e-03, -6.84420526e-04, -6.92471720e-03, 1.01683389e-02,
 5.64497516e-03, 1.37156492e-02, -8.87964158e-04, 7.96353776e-03,
 1.11990980e-02, -9.59546477e-03, 7.64451909e-03, 1.09021488e-02,
 7.70817372e-03, -3.16689832e-03, -1.13103460e-02, 4.89821383e-04,
 -1.33650384e-02, -3.58226912e-03, 1.35724967e-03, 1.11392334e-02,
 1.14413664e-02, 4.51247943e-03, 1.03163684e-02, -3.69510462e-03,
 1.39445779e-02, 1.27149151e-02, 1.13435549e-02, 5.10336831e-03,
 1.34319223e-02, 1.16372159e-02, 7.69728030e-03, 6.81205412e-03,
 -2.10710317e-03, -2.57812675e-03, -2.30579377e-03, -7.80003091e-04,
 1.28915177e-02, 1.41151062e-02, 2.10148668e-02, 5.27655912e-03,
 3.76281537e-03])